Holdline design and operating model

Version 2.0.0 · 26 October 2026

Holdline is a learner-facing receiving review with a complete local business path, a durable ledger, queued processing, and reproducible deployment checks. Its purpose is to make service contracts and failure handling visible. The records are synthetic and every result leaves stock on hold for a person.

Outcome and scope

The application reviews HL-2201 through HL-2204. Policy checks age and excluded categories. Shipment checks documents, damage, and whether freight was already received. Disposition combines those findings into one result. A clerk can inspect the evidence and audit record in the browser or through HTTP.

The project includes a synchronous API, queued submission and status, SQLite/Postgres persistence, migrations, backup tooling, a container deployment, and optional AWS infrastructure. The first run needs only Python. Optional model wording adds a provider dependency without transferring decision authority to it.

Human approval, warehouse execution, real customer data, retrieval, and per-user authorization are outside the receiving desk's scope. These are product boundaries, not implied completed features.

Service boundaries

Service	Responsibility	Interface
API	Authenticate requests, assign identity, orchestrate reviews, expose the desk	HTTP on 8741
Policy	Apply hold-window and category rules	A2A on 8743
Shipment	Apply document, damage, and receipt rules	A2A on 8744
Disposition	Combine validated findings and optionally explain them	A2A on 8745
Shipment tool	Return one typed synthetic shipment record	MCP on 8746
Receiving	Validate and persist reviews and job state	HTTP on 8742
Worker	Consume queued reviews and retry dependency failures	Celery over Redis

The API asks policy and shipment concurrently, then asks disposition. Both evidence specialists fetch the record through MCP. The case runner validates all messages before sending a final review to receiving. Receiving validates the write again before committing it.

Three service processes are useful here because learners can observe discovery and independent failures. A small real application may be better as one process. The separation adds network, deployment, and operating costs and should follow an actual ownership or scaling need.

Decisions and evidence

All five policy settings are required. Unknown keys, duplicates, invalid booleans, and invalid ranges reject the corpus. Shipment records are typed too. A missing policy never becomes a disabled check.

The hold window is inclusive: age 14 is within a 14-day window and age 15 is outside it. Rule precedence is:

1. Either finding missing, invalid, or ERROR: INCOMPLETE.
2. Otherwise, either finding BLOCK: BLOCKED.
3. Otherwise, both CLEAR: READY_FOR_REVIEW.

ERROR takes precedence over BLOCK because an incomplete review is not a complete assessment. None of these results is a release instruction.

Each completed finding includes its role, shipment ID, review ID, source names, the policy and shipment snapshots, and SHA-256 digests of their canonical JSON. The contract recalculates those digests and checks the verdict against the snapshots. The review requires both findings to share identity and evidence. The runner rejects a disposition response that changes the evidence. Snapshot differences between evidence specialists become incomplete evidence rather than a combined decision from two versions.

The hashes identify content; they are not digital signatures. The shared service credential and private service network establish the deployment trust boundary. A multi-tenant product needs scoped identities and a different authorization design.

Review identity and retries

An `Idempotency-Key` identifies one request's shipment and mode. The API maps it to a stable UUID review ID. Without a key, it creates a fresh UUID. A new review of the same shipment is allowed; a retry of the same review returns its saved result.

Receiving stores a request fingerprint and enforces uniqueness on review ID. Conflicting use of an identity returns 409. Transactional identity locking covers both synchronous reviews and queued jobs. Concurrent duplicates can repeat computation, but they share one committed ledger result. The runner checks for a saved review before making specialist calls.

An HTTP timeout is not proof that a write failed. Clients keep the same key while retrying or query the saved review. The API supplies the review ID in a response header for diagnostics.

A2A and MCP

`specialists/server.py` uses the pinned A2A SDK for agent cards, JSON-RPC tasks, artifacts, and streaming completion status. Cards advertise the role's configured URL and bearer scheme. `specialists/protocol.py` resolves the card, requests streaming, and requires a completed task with a data artifact. Failed or incomplete streams are evidence failures.

The SDK's task store is in memory. Restarted A2A tasks cannot be resumed. Recovery occurs at the application review level through stable identity and resubmission; the project does not promise durable A2A stream resumption.

`mcp_tools/shipment_server.py` exposes `get_shipment` using streamable HTTP. Every MCP route except health requires the token. The client checks tool discovery and validates returned records

before applying a rule. Protobuf JSON-number normalization is confined to the A2A transport so domain integer validation stays strict.

Model wording

The optional LangChain adapter receives the validated findings, source identifiers, selected disposition, and no-release state. It returns a bounded structured evidence summary. The program adds its own release and handoff statement. The model cannot modify the review contract.

Rehearsal constructs no model client. Live configuration requires the provider key and model name; LangSmith tracing is an independent opt-in. Provider exceptions, deadlines, and release/approval language use a deterministic evidence-based fallback labelled `explanation_source: fallback`. The disposition is still recorded.

Schema checks and text guards do not establish factual correctness of arbitrary prose. The evaluation guide requires comparison with the stored evidence. The clerk always sees the deterministic disposition and can inspect the snapshots.

Persistence and schema changes

SQLite supports the one-machine first run. PostgreSQL supports the container deployment and shared receiving service. Only receiving writes the business ledger and job registry. Rows contain review identity, shipment, disposition, release state, full payload, request fingerprint, and timestamp. The API exposes timestamps and bounded pagination.

Versioned migrations preserve legacy rows. Existing records get legacy identities; a timestamp backfilled at migration time is not their historical creation time. Migrations run transactionally with a database-specific lock. Backup/restore exercises use a separate destination and verify the saved evidence before switching the active path.

No vector extension or embedding dependency is needed for these small structured policies.

Queued work

`POST /jobs` reserves the identity in the receiving database, publishes a Celery task, and returns 202 with a status URL. The worker marks it running, invokes the same case runner as the synchronous route, and records completion. The ledger result is authoritative if the worker stops between the business commit and its status update.

Database reservation and broker publication are separate commits. A process failure between them can leave a pending job without a message. The defined recovery is to resubmit the same request with its original key. Publication is repeatable and the final write is idempotent. This design teaches the delivery gap explicitly; it does not claim an atomic outbox dispatcher.

Celery uses late acknowledgments, rejection on worker loss, bounded dependency retries, JSON messages, and a thread pool for network-bound reviews. Redis persistence and the receiving job registry allow a stopped worker to resume queued work. Status does not depend on Celery's expiring result backend.

Failure behavior

Failure	Behavior
Invalid policy or fixture at readiness	Service is unready
Policy or shipment unavailable	Error finding; disposition may record INCOMPLETE
Disposition unavailable or invalid	HTTP 503; no result is posted
Receiving unavailable	HTTP 503; retry with the same identity
Provider wording unavailable	Labelled fallback; deterministic result is preserved
Broker publication unavailable	HTTP 503; pending reservation remains recoverable
Key reused for another request	HTTP 409; original result remains unchanged
Review exceeds 55 seconds	HTTP 503; client retries with the original key

The Python supervisor stops the whole local stack on a child exit. Containers run independently, so degraded-service behavior can be exercised there. Readiness uses HTTP 503 for unavailable dependencies and does not mistake a present model name for a verified provider connection.

Access and operation

The HTML page, readiness, and health endpoints are public. Specialist discovery cards are public. Business read/write routes and the API schema require a shared bearer token. Tokens travel in headers and are compared in constant time. Log events include UTC time and review identity without authorization values.

The complete Compose stack exposes only the loopback API port. Its TLS overlay replaces that publication with a Caddy gateway. AWS uses a public HTTPS load balancer and private service tasks, encrypted authenticated Redis, and certificate-verified Postgres. Service-to-service HTTP remains private in these deployments; service-mesh identity is not implemented.

Use `docs/operations.md` for startup, migration, backup, rotation, and queue recovery. Use `docs/api.md` for exact request contracts and `docs/learner-guide.md` for the exercises.

Verification and release

Fast tests cover policy boundaries, complete verdict precedence, message tampering, identity conflicts, concurrent writes, legacy migration, recovery, authentication, and model-adapter behavior without paid calls. The integration script exercises the actual A2A and MCP processes, all four cases, failures, and recovery. Optional flags add PostgreSQL and real queued processing. CI runs these checks and the container acceptance path.

Word and PDF exports are generated from this source and visually reviewed. `docs/release-validation.md` records checks performed. Release archives include a file-hash manifest and exclude local credentials, data, caches, and virtual environments. Publishing and deploying remain maintainer actions.